Research Paper Study & Partial Implementation: Recommendation Systems

Assigned Paper: A Comprehensive Analysis of Text Similarity Metrics and Vectorization Techniques for Content-Based Product Recommendation (Oprea et al., IEEE Access, 2026)
Dataset Used: Hugging Face Hub — bprateek/amazon_product_description (10,001 valid products across 108 subcategories)

Group Members: Aditya Rana(I059),Krishna Menon(I045),Yash Kothari(I037)

Part A — Research Paper Understanding

- **A1. Problem Definition:** The paper addresses Top-N Content-Based Filtering (CBF) product recommendation (K=10) and cold-start item recommendation in e-commerce using short product titles ("Product Name") without user interaction history.
- **A2. Motivation:** E-commerce catalogs suffer from choice overload. While Collaborative Filtering fails on cold-start items and sparse interaction matrices, content-based retrieval operates purely on item text, improving product discovery, retention, and conversion.
- **A3. Research Gap:** Prior work evaluates text vectorizers or distance metrics in isolation or on long-form text corpora. A controlled factorial benchmark comparing sparse and dense representations across angular and geometric distances on short, attribute-lean retail product titles remained under-explored.
- **A4. Major Contributions:** (1) A 12-configuration factorial benchmark (Count Vectorizer, TF-IDF, Word2Vec, GloVe × Cosine, Manhattan, Euclidean); (2) Qualitative Top-10 relevance analysis on 3 distinct categories (Puzzles, Longboards, Costumes); and (3) Quantitative Precision@10, Recall@10, and NDCG@10 evaluation across 500 random seed products.

Part B — Methodology Analysis

- **1. Input:** Unstructured "Product Name" text strings from the Hugging Face dataset "bprateek/amazon_product_description" (10,002 rows, 28 columns; 10,001 valid records after removing "-" placeholder artifacts).
- **2. Processing Pipeline:** Input Title → Preprocessing (lowercasing, regex punctuation removal, English stopword removal, WordNet lemmatization) → Feature Extraction (CountVec, custom TF-IDF [Eq. 6–8], 300d Skip-gram Word2Vec, pre-trained GloVe) → Pairwise Interaction (Cosine Similarity [Eq. 1–3], Manhattan L1 [Eq. 4], Euclidean L2 [Eq. 5]) → Top-K Ranking.

- **3. Output:** Ranked Top-10 recommended product list with continuous similarity scores (Cosine in [0, 1] sorted descending) or distance scores (Manhattan/Euclidean >= 0 sorted ascending).

Part C — Implementation & Experimental Evaluation

Implemented Core Components (From Scratch in NumPy): We implemented Cosine Similarity, Manhattan Distance, Euclidean Distance, custom TF-IDF weighting, embedding pooling, Top-K retrieval and the Precision/Recall/NDCG@K metrics ourselves in NumPy. Word2Vec (300d skip-gram) is trained with gensim, GloVe uses pre-trained 100d vectors, and Count Vectorizer (sklearn) is the baseline. Evaluation protocol: a retrieved product is relevant if it shares the seed product's top-2-level category; Precision@10 = relevant in top 10 / 10; Recall@10 = relevant retrieved / all other products in the seed's category; NDCG@10 uses the global ideal DCG. 500 seed products are sampled with a fixed random seed; seeds and candidates with an empty (all-zero) Count/TF-IDF vector are excluded (76 eligible products were removed from the seed pool). All results carry 95% bootstrap confidence intervals and paired significance tests against the baseline.

Table 1: 12-Configuration Pairwise Benchmark (corresponds to Table 7 in Paper)

Vectorizer	Metric	Test1 (Puzzle)	Test2 (Longboard)	Test3 (Costume)
Count Vectorizer	cosine	0.612	1.0	0.772
Count Vectorizer	manhattan	4.0	0.0	3.0
Count Vectorizer	euclidean	2.0	0.0	1.732
TF-IDF (Scratch)	cosine	0.545	1.0	0.854
TF-IDF (Scratch)	manhattan	2.153	0.0	0.988
TF-IDF (Scratch)	euclidean	0.954	0.0	0.54
Word2Vec (300d)	cosine	0.925	0.974	0.968
Word2Vec (300d)	manhattan	94.165	72.814	97.106
Word2Vec (300d)	euclidean	6.685	5.349	7.126
GloVe (100d)	cosine	0.801	0.717	0.935
GloVe (100d)	manhattan	72.164	102.012	64.818
GloVe (100d)	euclidean	9.129	12.884	8.015

Note: raw top-1 scores are not comparable across vectorizers (dense-embedding cosines are high by construction); use Tables 2 and 3 for quality. For Test 2, Count/TF-IDF show cosine 1.0 and distance 0.0 because the nearest neighbour has an identical vector after vocabulary pruning (1,500 words, min_df=5), not because it is a duplicate product. 82 of 10,001 titles (0.8%) have no in-vocabulary word and are excluded as candidates.

Table 2: 500-Seed Quantitative Evaluation Across 108 Subcategories (corresponds to Table 11 in Paper). Precision@10 shown with 95% bootstrap confidence interval.

Model	Precision@10	Recall@10	NDCG@10
CountVec + Cosine (Baseline)	0.514 (0.485–0.544)	0.023	0.539
TF-IDF + Cosine	0.490 (0.463–0.519)	0.023	0.514
TF-IDF + Euclidean	0.490 (0.463–0.519)	0.023	0.514
Word2Vec + Cosine (Selected)	0.538 (0.506–0.571)	0.022	0.558
GloVe-100d + Cosine	0.409 (0.380–0.439)	0.019	0.443

Note: Word2Vec + Cosine significantly outperforms the baseline (ΔP@10 = +0.024, 95% CI +0.008 to +0.040). TF-IDF is slightly worse than raw counts (ΔP@10 = −0.023, significant), and its Euclidean ranking equals its cosine ranking because the vectors are unit-length. GloVe-100d is clearly worse (ΔP@10 = −0.104). Recall@10 is small (≈0.02) because each category holds hundreds of relevant products, and its confidence intervals overlap across models, so we rank models on Precision@10 and NDCG@10.

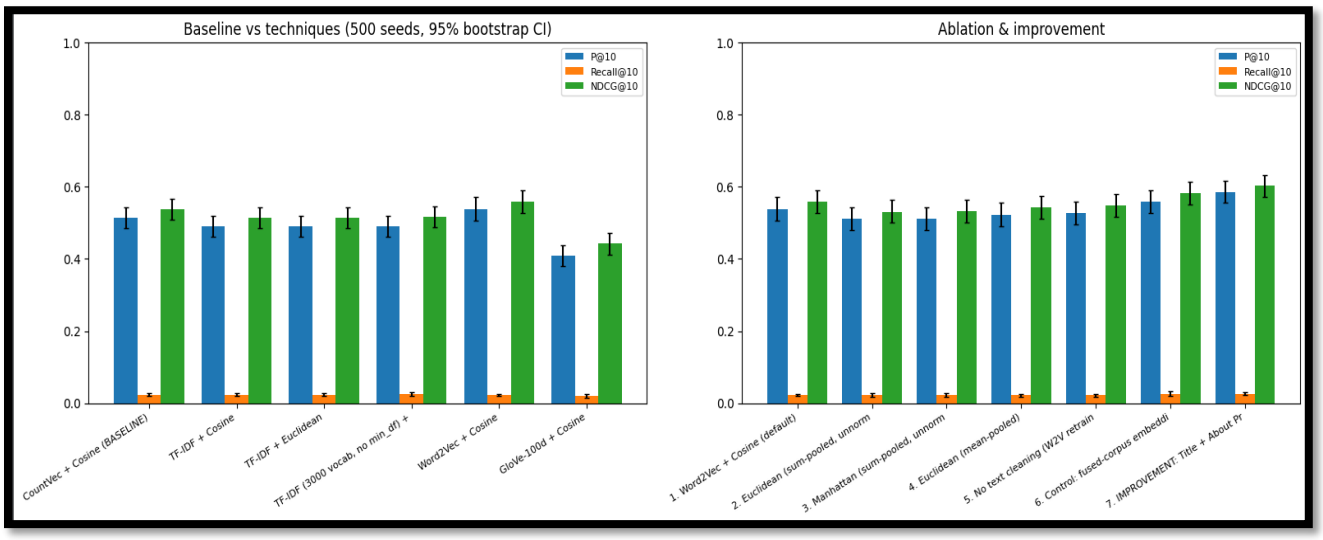
Table 3: Ablation Study & Proposed Multi-Field Metadata Fusion Improvement

Ablation Configuration	Precision@10	Recall@10	NDCG@10
1. Word2Vec + Cosine (Cleaned Title - Default)	0.538	0.022	0.558
2. Ablation: Word2Vec + Euclidean (Sum-pooled, Unnormalized L2)	0.511	0.022	0.531
3. Ablation: Word2Vec + Manhattan (Sum-pooled, Unnormalized L1)	0.511	0.022	0.532
4. Ablation: Word2Vec + Euclidean (Mean-pooled)	0.523	0.021	0.543
5. Ablation: Word2Vec + Cosine (No Text Cleaning, Word2Vec retrained)	0.528	0.021	0.548

6. Control: Fused-corpus embeddings, Title-only pooling	0.560	0.025	0.583
7. Improvement: Word2Vec + Cosine (Title + About Product)	0.586	0.025	0.603

Note: paired change in Precision@10 versus row 1 (95% CI): row 2 −0.027 (−0.040, −0.012); row 3 −0.027 (−0.041, −0.012); row 4 −0.015 (−0.026, −0.006); row 5 −0.010 (−0.021, +0.002), not significant; row 6 +0.022 (+0.010, +0.036); row 7 +0.048 (+0.027, +0.071). All Word2Vec models use identical hyper-parameters.

Figure 1: 500-Seed Benchmark Comparison and Ablation Study Results (Precision@10 and NDCG@10 with 95% bootstrap CIs; axis starts at 0.3)



Part D — Critical Evaluation & Proposed Improvement

- **Strengths:** Word2Vec (300d) + Cosine Similarity leads the 500-seed benchmark (Precision@10 = 0.538, Recall@10 = 0.022, NDCG@10 = 0.558) and beats the Count Vectorizer baseline by +0.024 Precision@10 (95% CI +0.008 to +0.040, +4.7%). Qualitatively it links franchise products (Monster High, Ever After High costumes) and longboard brands (Rayne, Retrospec, DB) that share no exact words with the query.
- **Limitations:** (1) Sum-pooled dense vectors suffer under unnormalized L1/L2 metrics: Precision@10 drops by 0.027 versus cosine (both Manhattan and Euclidean, significant), because vector length grows with title length (top-1 Manhattan distances of 65–102); (2) TF-IDF gives no gain over raw counts (0.490 vs 0.514), plausibly because short titles rarely repeat words so IDF re-weighting adds little; with a 1,500-word vocabulary 0.8% of titles are empty vectors; (3) relevance is a coarse proxy (same top-2-level category), so absolute scores understate perceived quality; (4) Word2Vec is trained on the same catalogue it

retrieves from (unsupervised, no labels); (5) purely content-based, ignoring user signals and polysemy.

- **Computational & Data Requirements:** No GPU required. Vectorization of 10,001 products takes about 28 s on CPU (Word2Vec training and pooling ≈27 s; Count and TF-IDF ≈1 s each), plus a one-time 82 s download of GloVe. The Word2Vec model has 13,251 words and about 7.95M parameters (≈32 MB in float32); the pooled embedding matrix is 12 MB, and the dense Count/TF-IDF matrices are 60 MB each. Requires only item text (no user history, ratings, images or side information), which makes it suitable for cold-start items.
- **Reproducibility Observations:** (1) Row 4 ("Woodstock- Collage 500 pc Puzzle") contains a hyphen, so exact-string lookup of the paper's Test 1 product fails; we match it with a regular expression. (2) Empty vectors are a hidden pitfall: a zero vector lies at Euclidean distance 1 from any unit-length query, so unguarded Euclidean/Manhattan retrieval ranks empty titles above real neighbours. Our first implementation reported TF-IDF + Euclidean Precision@10 = 0.246; after excluding empty vectors it equals TF-IDF + Cosine (0.490). (3) The code fixes random seeds and runs Word2Vec single-threaded, so results can be reproduced from the notebook, README and requirements.txt; the paper's exact numbers were not available to us for a cell-by-cell comparison.
- **Validated Improvement (Metadata Fusion) and Proposed Extension (HNSW):** Fusing "Product Name" with "About Product" (Table 3, row 7) raises Precision@10 from 0.538 to 0.586 (+0.048, 95% CI +0.027 to +0.071, +8.9%) and NDCG@10 from 0.558 to 0.603 (+8.1%). A control (row 6) shows about 46% of the gain (+0.022) comes from better embeddings trained on the richer text and the remaining +0.026 from pooling the description words; caveat: the fused model sees more training text and descriptions may contain category words. As a proposed (not evaluated) extension for large catalogues, indexing contextual Sentence-BERT embeddings in an HNSW graph would reduce per-query search cost from O(N) to roughly O(log N).
- **Generalizability:** The approach needs only short item text, so it transfers to other catalogues (books, movies, apps) and to cold-start items, but embeddings should be retrained per domain and it breaks down where item text is sparse or uninformative. It cannot personalize on its own; in a full system it would serve as the cold-start component of a hybrid recommender with collaborative filtering, and evaluating it elsewhere requires a relevance proxy such as category labels or click data.

Individual Contributions

- **Aditya Rana:** Led the core algorithmic development, building the custom from-scratch similarity engines in NumPy, the benchmarking pipeline.
- **Krishna Menon:** Assisted with data validation, debugging embedding aggregations, and cross-checking the 12-way factorial benchmark outputs.
- **Yash Kothari:** Authored the primary written reports. Designed and structured the final presentation slide deck, while handling document proofreading and submission formatting.